WAKE GUARD

Comprehensive Project Report & Technical Architecture

Team META MINDS

Role	Name
Team Lead	Mohd Junaid Pasha
Member	Mohd Saif Patel
Member	Farjana Shaikh
Mentor	Dr. K Sampath

1. Executive Summary

WakeGuard is an innovative, dual-purpose safety application designed to prevent users from falling asleep in critical situations, whether they are operating a motor vehicle or commuting on public transport. Stemming from the `Iamjunade/wakeguard` repository, the core capability of the system currently resides in real-time, AI-powered computer vision—specifically Driver Drowsiness Detection using facial landmarks and Eye Aspect Ratio (EAR) calculations. However, addressing the pervasive problem of "sleeping past your stop" on public transport introduces a location-based paradigm. This report synthesizes the analyzed state of the repository with **Industry Standard Best Practices [Recommended/Assumed]** to present a unified architecture combining visual drowsiness detection and GPS-based geofencing alarms.

The unique value proposition of WakeGuard lies in its versatility. By integrating MediaPipe Face Mesh for browser-based eye tracking, OpenCV and dlib for robust desktop execution, and an assumed geolocation pipeline for transit alerts, the application transitions from a simple timer to a context-aware safety net. The current implementation successfully executes facial tracking at over 15 FPS in the browser, triggering a Web Audio API alarm and HTTP SMS alerts. When augmented with a Geofencing Engine, WakeGuard demonstrates high technical viability, offering a seamless, low-latency, and high-reliability solution for commuter and driver safety.

---

2. Introduction

Project Background

The conceptualization of WakeGuard originated from two parallel safety concerns: the fatal risks associated with drowsy driving, and the chronic inconvenience/vulnerability of commuters falling asleep on public transit and missing their destinations. The development team (META MINDS, under the guidance of Dr. K Sampath) initiated the project to create an accessible, zero-friction application that requires no specialized hardware beyond a standard webcam and GPS-enabled smartphone.

Problem Statement

A significant percentage of public transit riders experience fatigue, resulting in "sleeping past your stop." This not only leads to wasted time and increased travel costs but also poses security risks for individuals waking up in unfamiliar or unsafe transit terminals late at night. Conversely, drivers on long-haul trips face life-threatening consequences from microsleeps. WakeGuard addresses these overlapping problems by providing automated, context-aware wake-up triggers based on physical state (eyes closed) and geographic state (approaching a destination).

Objectives and Scope

The primary objectives of the WakeGuard application include:

1. **Real-Time Drowsiness Detection:** Monitor a user's eye aspect ratio (EAR) using webcam feeds to detect sleep onset within 2 seconds.
2. **Geofenced Location Alarms [Recommended/Assumed]:** Allow users to set a destination radius, triggering high-decibel alarms when crossing the boundary.
3. **Multi-Platform Accessibility:** Deliver the solution across Desktop (Python/OpenCV) and Web (HTML/JS/MediaPipe).
4. **Emergency Escalation:** Send automated SMS notifications via the HTTPSMS API to designated contacts if the user fails to respond to alarms.

---

3. System Architecture & Design

High-Level Architecture

The system architecture follows a decoupled client-server model, though the majority of the processing (both computer vision and location tracking) is optimized for edge execution (client-side) to preserve battery and reduce latency.

1. **Frontend / Edge Client:** Houses the Camera processing pipeline (MediaPipe/dlib) and the GPS/Location Tracker. The frontend continuously polls the HTML5 Geolocation API (or native GPS modules in mobile environments) and compares the current coordinate vector against the destination vector.
2. **Notification Engine:** Interacts with the browser's Web Audio API for local alarms and triggers external POST requests to the HTTPSMS REST API for remote alerting.
3. **Backend / Storage [Recommended/Assumed]:** While the current repo is entirely static/client-side, an assumed backend (Node.js/Express) manages user profiles, saved transit routes, and OAuth authentication.

Tech Stack Breakdown

Currently Implemented in Repository:

- **Languages:** Python 3.7+, JavaScript (ES6+), HTML5, CSS3.
- **Computer Vision Libraries:** `opencv-python`, `dlib`, `imutils` (Desktop); `@mediapipe/face_mesh` (Web).
- **Math & Geometry:** `scipy.spatial.distance` (for Euclidean calculations).
- **Alerting APIs:** HTTPSMS API, Web Audio API, `pygame.mixer` (for desktop audio).
- **Deployment:** Vercel (Web representation).

Assumed / Recommended Stack Extensions for Location Alarms:

- **Mobile Framework:** React Native or Expo (to port the web implementation to iOS/Android with native background location access).
- **Location Services:** Google Maps API (for Places autocomplete) and Native Geolocation APIs (`navigator.geolocation.watchPosition`).
- **Testing Suites:** Jest (JavaScript unit testing), Mocha, and Appium (for mobile E2E testing).
- **State Management:** Redux or React Context API to manage active Geofence states and UI overlays.

Key Modules

1. "Set Destination" Module [Recommended/Assumed]

This module governs user input for geolocation. A user types a destination (e.g., "Central Station"), which is resolved to latitude and longitude coordinates via the Google Maps Geocoding API. The user then defines an "Alarm Radius" (e.g., 500 meters). This data is stored in the application's global state.

2. "Distance Calculation" Module [Recommended/Assumed]

Once a destination is set, the application requests continuous GPS updates. A background worker thread receives pairs of coordinates (User_Lat, User_Lon) and calculates the spherical distance to (Target_Lat, Target_Lon) every 10 seconds. If the distance is less than the Alarm Radius, an event is emitted to the Alarm Trigger.

3. "Alarm Trigger" Module (Implemented)

This module acts as the sink for multiple detection events. In the codebase, it is represented by the `triggerAlarm()` function in `app.js`. When the EAR threshold is breached (or assumed distance breached), the system oscillates a Web Audio API square wave at 800Hz and invokes `sendSmsAlert()` to notify contacts.

---

4. Technical Implementation

Geofencing Logic (Distance Calculation)

Geofencing relies on accurately determining the distance between two points on the Earth's surface. Since the Earth is a sphere, simple Pythagorean geometry is insufficient. **[Recommended/Assumed]** WakeGuard utilizes the **Haversine Formula** to calculate the great-circle distance between the user's current GPS location and the target transit stop.

The mathematical formulation implemented in the assumed JavaScript module is:

```
function calculateHaversineDistance(lat1, lon1, lat2, lon2) {
    const R = 6371e3; // Earth's radius in meters
    const φ1 = lat1 * Math.PI/180; // φ, λ in radians
    const φ2 = lat2 * Math.PI/180;
    const Δφ = (lat2-lat1) * Math.PI/180;
    const Δλ = (lon2-lon1) * Math.PI/180;

    const a = Math.sin(Δφ/2) * Math.sin(Δφ/2) +
              Math.cos(φ1) * Math.cos(φ2) *
              Math.sin(Δλ/2) * Math.sin(Δλ/2);
    const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));
    const distance = R * c; // Distance in meters

    return distance; // Alarm triggers if distance < threshold
}
```

Computer Vision Implementation (Repository Code)

The repository heavily leverages facial tracking logic in `drowsiness_detect.py` and `app.js`. In the Python application, the calculation uses the `scipy.spatial.distance.euclidean` function. The 68 facial landmarks are predicted by dlib (`shape_predictor_68_face_landmarks.dat`).

```
# From drowsiness_detect.py
def eye_aspect_ratio(eye):
    A = dist.euclidean(eye[1], eye[5])  # Vertical distance 1
    B = dist.euclidean(eye[2], eye[4])  # Vertical distance 2
    C = dist.euclidean(eye[0], eye[3])  # Horizontal distance
    ear = (A + B) / (2.0 * C)
    return ear
```

The threshold (`CONFIG.EAR_THRESHOLD = 0.22` in JS, `0.25` in Python) dictates when eyes are closed. If the eyes remain closed for `CONSEC_FRAMES_THRESHOLD` (approx. 2 seconds), the `triggerAlarm()` sequence executes.

API Integrations

The `package.json` and equivalent scripts reveal the absence of heavy backend frameworks, opting for lightweight Fetch API calls. The **HTTPSMS API** is deeply integrated into `app.js`:

```
// From app.js
const response = await fetch('https://api.httpsms.com/v1/messages/send', {
    method: 'POST',
```

```
    headers: {
        'x-api-key': CONFIG.HTTPSMS_API_KEY,
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        from: CONFIG.HTTPSMS_SENDER,
        to: CONFIG.SMS_RECIPIENT,
        content: '⚠️ WAKEGUARD ALERT: Drowsiness detected! Please take a break and rest.'
    })
});
```

---

5. UI/UX Design

Interface Analysis

Based on the Web Application structure (`wakeguard-web/index.html` and `style.css`), the interface is minimalist, optimizing for immediate action.

1. **The Viewport:** The user interface features a centered video feed encapsulated in a rounded-corner container.
2. **Heads Up Display (HUD):** Overlay elements display the raw EAR value (`earValueElement`) and FPS (`fpsValueElement`), providing immediate technical feedback to the user regarding tracking accuracy.
3. **Status Badges:** A glowing `.status-dot` acts as a pulsating indicator of active tracking, turning green for normal operation and flashing red (`.alert` class) upon threshold breaches.

User Flow

1. **Onboarding:** The user accesses `wakeguard.vercel.app` (or the mobile app). They are presented with a "Ready - Click Start" state.
2. **Configuration:** To configure SMS notifications, the app contains an easter egg ("Secret Settings"). The user types `pasha123` , a secret code defined in `app.js` , to reveal a modal dialogue for configuring a phone number. **[Recommended/Assumed]** For the location workflow, this is where the user interacts with a Maps autocomplete field to input their transit stop.
3. **Execution:** The user clicks "Start Detection". The browser requests camera access (and geolocation permissions). A loading overlay handles asynchronous MediaPipe instantiation.
4. **Alarms:** Upon arriving at the destination radius or falling asleep, an immersive screen-takeover overlay (`alertOverlay.classList.add('active')`) blasts the Web Audio alarm, ensuring maximum wakefulness.

---

6. Testing & Quality Assurance

Identified & Assumed Test Suites

The repository currently lacks a formalized testing directory. However, to maintain enterprise-grade reliability for an application triggering emergency SMS messages and critical location data, the following **[Recommended/Assumed]** test architecture must be implemented:

- **Jest:** Fast, snapshot-based unit testing for the Geofencing calculations (e.g., verifying `calculateHaversineDistance` returns `< 1%` margin of error).
- **Mocha & Chai:** For testing the asynchronous HTTPSMS API integrations, mocking network failures to ensure the app handles API limits (cooldown gracefully catches rapid requests).

- **Selenium/Puppeteer:** End-to-End (E2E) testing for verifying MediaPipe Face Mesh canvas renders across different browser configurations (Chrome, Firefox, Safari).

Location Accuracy & Battery Consumption Test Plan

Location tracking creates immense battery draw. A robust QA plan is essential:

1. **Location Polling Intervals:** Test varying the Geolocation `watchPosition` parameter based on velocity. If the user is 10 miles away, poll every 60 seconds; if 1 mile away, poll every 5 seconds.
2. **Battery Metrics:** Use Android Studio Profiler and Xcode Instruments to monitor mAh consumption. The target metric is <5% battery drain per hour while the app is in background tracking mode.
3. **Accuracy Degradation:** Test geofence triggers in low-signal areas (e.g., subway networks). The system should implement a "Last Known Good Location" fallback, predicting destination arrival times using transit schedules if GPS is lost underground.

---

7. Future Enhancements & Scalability

The fundamental computer vision capabilities of WakeGuard are exceptional, yet the product roadmap can scale significantly by implementing the aforementioned location architectures. Suggested enhancements include:

1. **Offline Maps Integration:** Public transit commuters frequently lose internet access in tunnels. Implementing Mapbox offline SDKs and on-device offline geocoding would ensure the location alarm fires even without a cellular connection.
2. **Battery Optimization via Activity Recognition:** By tapping into the iOS/Android `CMMotionActivityManager`, WakeGuard can pause heavy CPU processes (like 60 FPS Camera polling) when it detects the user is stationary or walking, re-engaging computer vision only when in a vehicle.
3. **Integration with Public Transit Schedules:** Connecting to global GTFS (General Transit Feed Specification) APIs would allow users to select "Take the 4 Train to Union Square". WakeGuard would inherently know the route and expected arrival time, cross-referencing this with GPS to provide highly accurate wake-up alerts even given service delays.
4. **Wearable Device Sync:** Porting the alarm triggers to Apple Watch and WearOS devices to deliver haptic feedback (vibrations) on the wrist. Haptic feedback is typically more effective at abruptly waking a user on loud public transport than audible alarms.

---

8. Conclusion

WakeGuard stands as a testament to the power of edge-computed artificial intelligence and context-aware sensing. By successfully implementing robust Eye Aspect Ratio parsing through MediaPipe and dlib, the founding META MINDS team has created a core detection engine with exceptionally low latency.

When fused with **Industry Standard Best Practices** for GPS Geofencing and Haversine spatial tracking, the system evolves into a comprehensive safety net for drivers and public transport commuters alike. Resolving the "sleeping past your stop" dilemma requires only a logical extension of WakeGuard's existing architecture—bridging the application from a physical state monitor to a geographic state monitor. With targeted testing frameworks (Jest/Appium) and strategic enhancements (Offline mapping, haptics), WakeGuard has the capacity to fundamentally transform commuter safety and transport reliability globally.